EXP-2	Build a simple CNN model for image segmentation

AIM:

- To develop a simple CNN-based model for segmenting an image.
- To understand encoder-decoder architecture for pixel-wise prediction tasks.

Algorithm:

1. **Import Libraries:** Import TensorFlow, Keras, NumPy, OpenCV, and matplotlib.
2. **Upload and Preprocess Image:**
    - Upload an image using Colab.
    - Resize to a fixed dimension (128x128) and normalize pixel values.
3. **Generate Synthetic Mask:**
    - Convert the image to grayscale.
    - Apply Otsu's thresholding to create a binary mask.
4. **Define CNN Model:**
    - Use an encoder-decoder structure with Conv2D, MaxPooling2D, UpSampling2D, and Conv2DTranspose layers.
5. **Compile Model:** Use Adam optimizer and binary crossentropy loss.
6. **Train Model:** Fit the model with the image and its corresponding mask.
7. **Predict and Visualize:**
    - Predict the segmented mask.
    - Apply a threshold and visualize the original image, mask, and predicted output.

Code:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import cv2
from google.colab import files
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, UpSampling2D, Conv2DTranspose
from tensorflow.keras.models import Model

# Upload an image
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Read and preprocess the image
image = Image.open(image_path).convert('RGB')
image = image.resize((128, 128))  # Resize to 128x128
image_array = np.array(image) / 255.0  # Normalize to [0,1]

# Convert image to uint8 before processing
image_array_uint8 = (image_array * 255).astype(np.uint8)

# Generate synthetic mask using thresholding
gray = cv2.cvtColor(image_array_uint8, cv2.COLOR_RGB2GRAY)
_, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
mask = mask.astype(np.float32) / 255.0  # Convert to binary mask [0,1]
mask = np.expand_dims(mask, axis=-1)  # Add channel dimension

# Define the CNN model
def build_segmentation_model(input_shape):
    inputs = Input(shape=input_shape)

    # Encoder
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(inputs)
    x = MaxPooling2D((2, 2))(x)
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = MaxPooling2D((2, 2))(x)

    # Decoder
```

```
    x = Conv2DTranspose(128, (3, 3), activation='relu', padding='same')(x)
    x = UpSampling2D((2, 2))(x)
    x = Conv2DTranspose(64, (3, 3), activation='relu', padding='same')(x)
    x = UpSampling2D((2, 2))(x)

    # Output layer
    outputs = Conv2D(1, (1, 1), activation='sigmoid')(x)

    model = Model(inputs=inputs, outputs=outputs)
    return model

# Build and compile the model
model = build_segmentation_model((128, 128, 3))
model.compile(optimizer='adam', loss='binary_crossentropy')

# Prepare data for training
X = np.expand_dims(image_array, axis=0)  # Add batch dimension
y = np.expand_dims(mask, axis=0)

# Train the model
history = model.fit(X, y, epochs=50, verbose=0)

# Predict the mask
predicted_mask = model.predict(X)[0]

# Threshold the prediction
predicted_mask_thresholded = (predicted_mask > 0.5).astype(np.float32)

# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(image_array)
plt.axis('off')

plt.subplot(1, 3, 2)
plt.title('Generated Mask (Ground Truth)')
```

```
plt.imshow(mask.squeeze(), cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title('Predicted Mask')
plt.imshow(predicted_mask_thresholded.squeeze(), cmap='gray')
plt.axis('off')

plt.show()

plt.show()
```

Output:

Result:

A simple encoder-decoder CNN model was implemented for image segmentation. The model successfully predicted a binary mask close to the ground truth.